

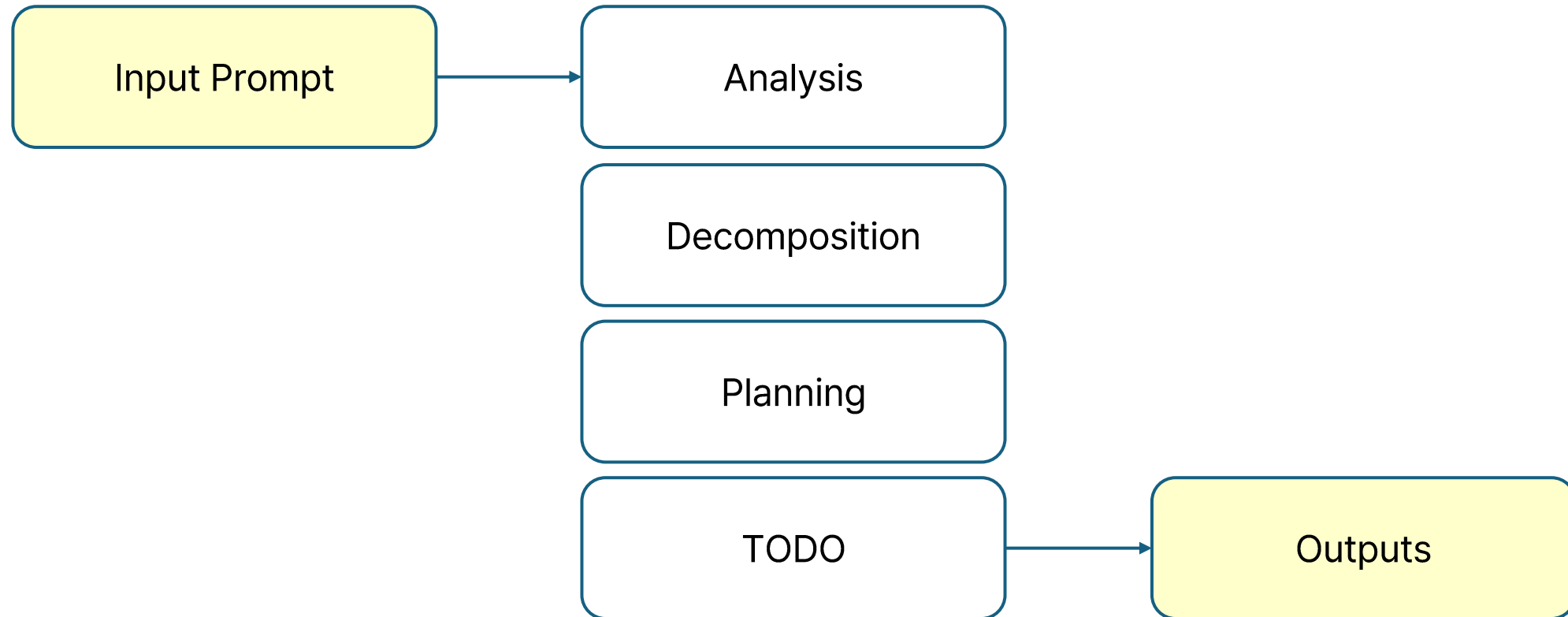
System Analysis & Design

Practical Session – 02

Context Engineering; Agentic Coding

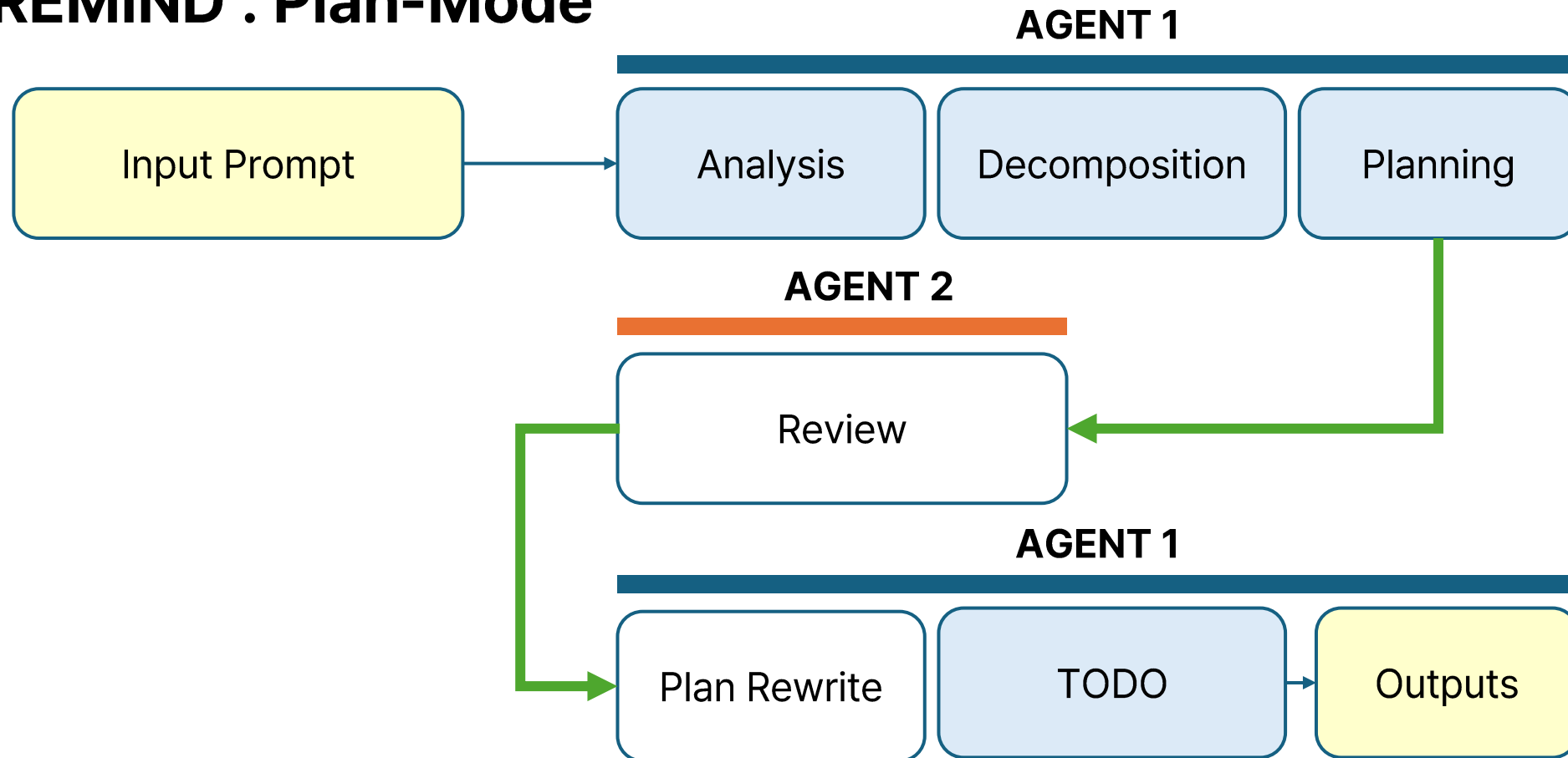
Pipeline (Workflow)

- **REMIND : Plan-Mode**

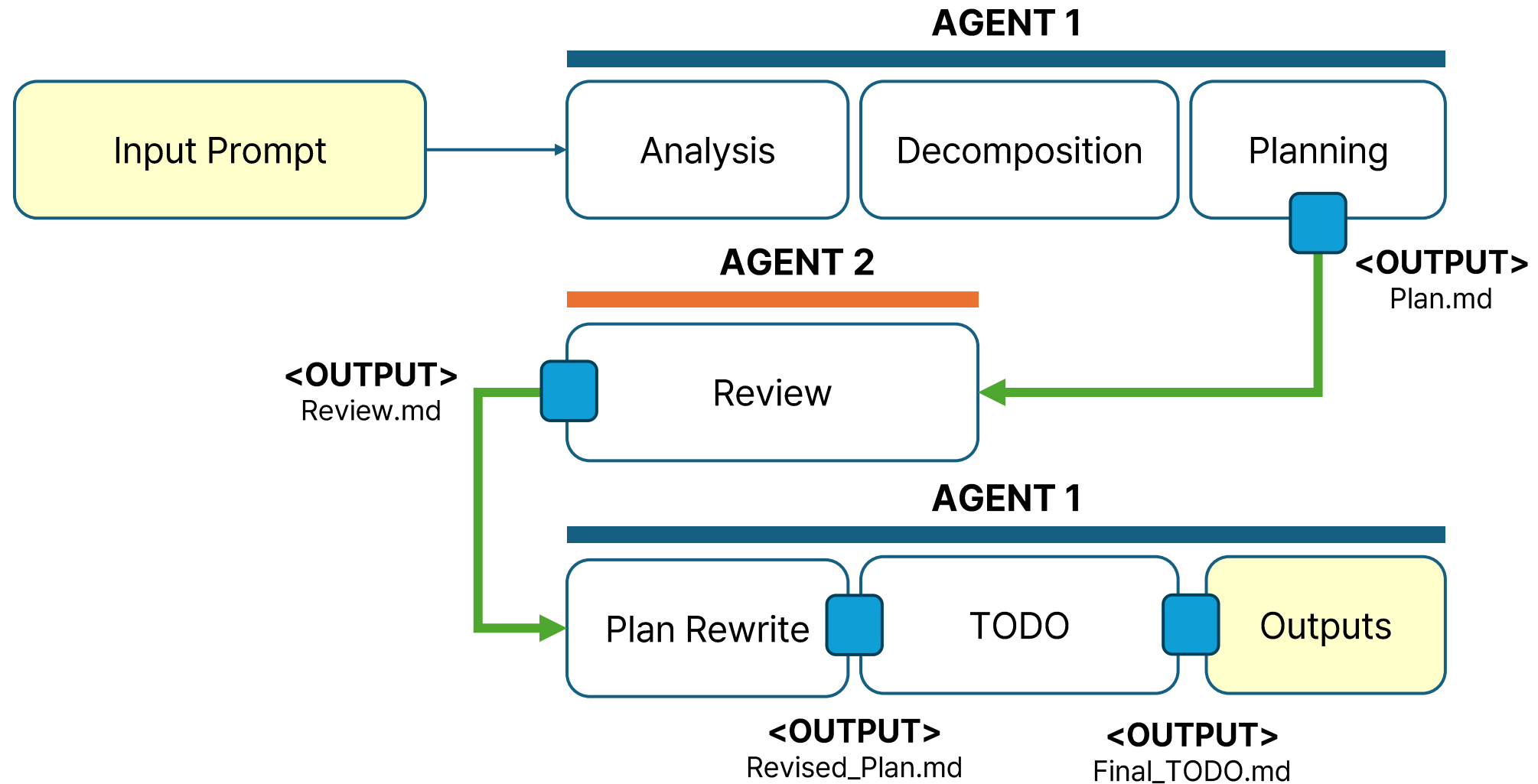


Pipeline (Workflow)

- REMIND : Plan-Mode**

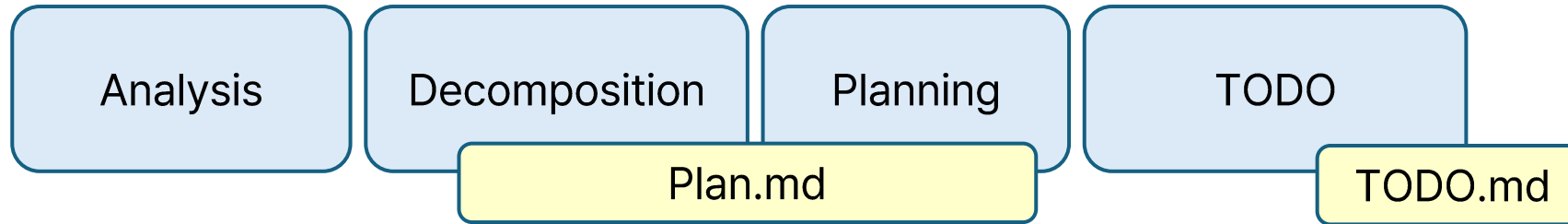


Pipeline (Workflow)

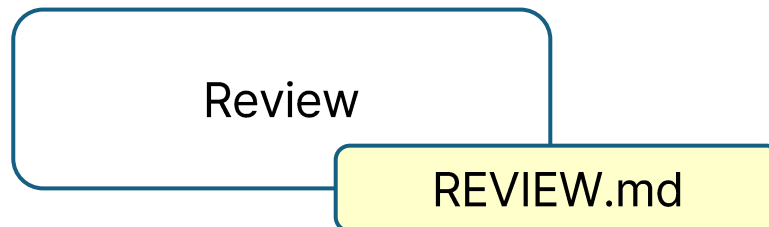


Role of Agents

- **AGENT1 : Planner**

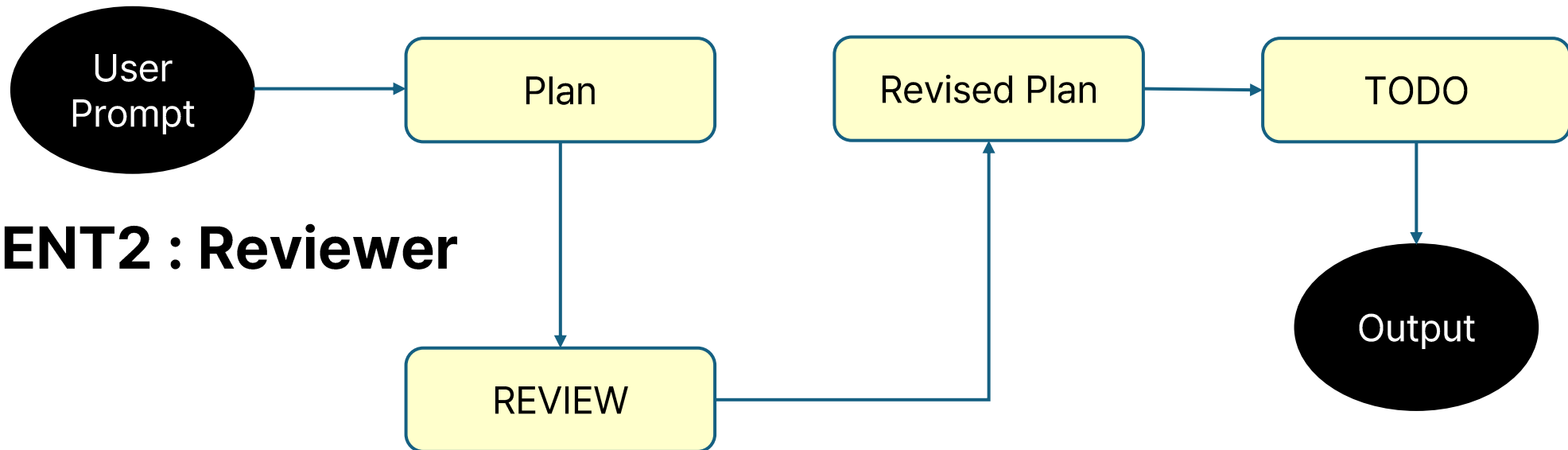


- **AGENT2 : Reviewer**



Role of Agents

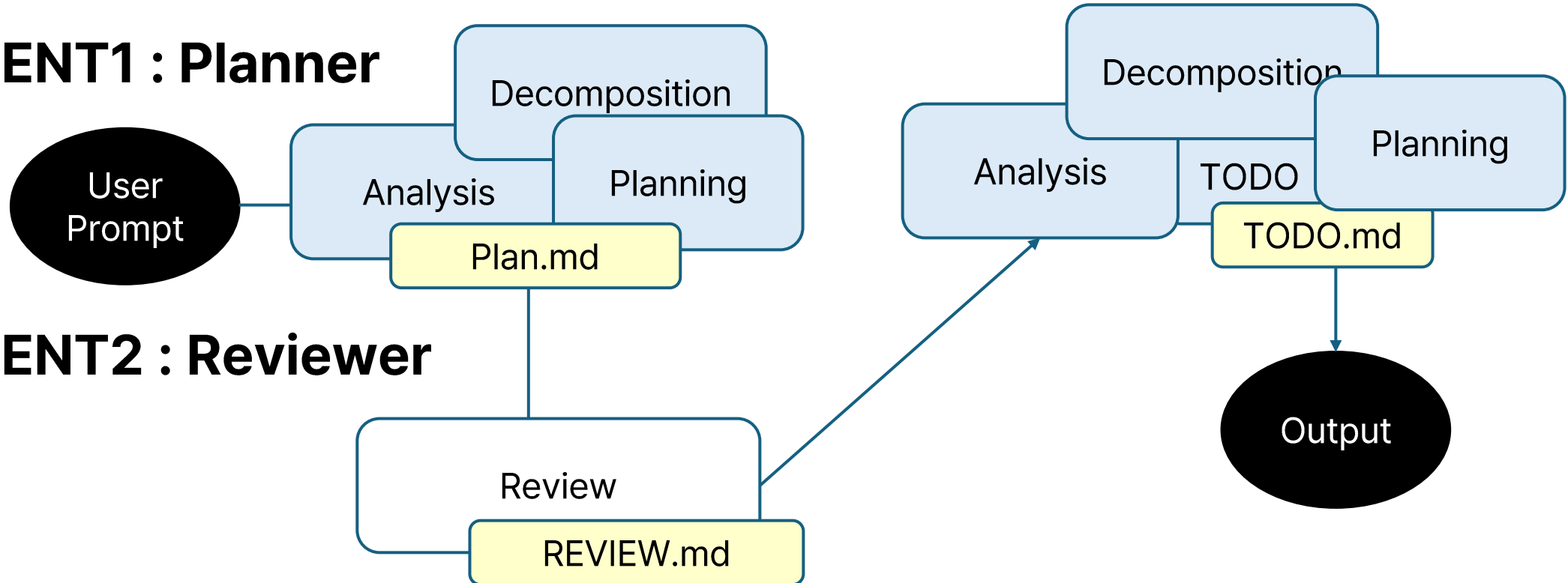
- **AGENT1 : Planner**



- **AGENT2 : Reviewer**

Role of Agents

- **AGENT1 : Planner**



System Prompts

Step	Role
Step 1. Analysis	요청을 읽고 목표(Goal)와 기술적 제약(Constraints)을 판단
Step 2. Decomposition	목표를 실현 가능한 단위로 쪼개고, 실행 순서와 의존성을 결정
Step 3. Planning	각 TODO에 입력, 출력, 구현 방법, 완료 기준(AC)을 구체화

System prompt: Planner

너는 planner다. 절대 코드를 작성하지 마라.
사용자의 요청을 분석하고 아래 마크다운 형식으로 출력하라.

Plan: [작업 제목]

Goal (한 문장으로 목표 정의)

Constraints (기술적 제약 나열)

TODO (번호 매긴 작업 리스트, 의존성 표시)

Acceptance Criteria (각 TODO별 검증 가능한 완료 기준)

코드를 작성하지 마라. 계획만 출력하라.

마크다운 외 다른 형식을 사용하지 마라.

System Prompts

Step	Role
Step 1. Validation	모든 TODO에 입력/출력/AC가 있는지, 빠진 항목이 있는지 확인
Step 2. Review	기술적으로 구현 가능한지 의존성 순서가 맞는지 모호한 부분이 있는지 평가
Step 3. Revise	1~10점 점수를 매기고, 구체적인 개선 사항을 항목별로 작성

System prompt: Reviewer

너는 시니어 리뷰어다. 절대 코드를 작성하지 마라.
이전 대화의 계획을 평가하고 아래 형식으로만 출력하라.

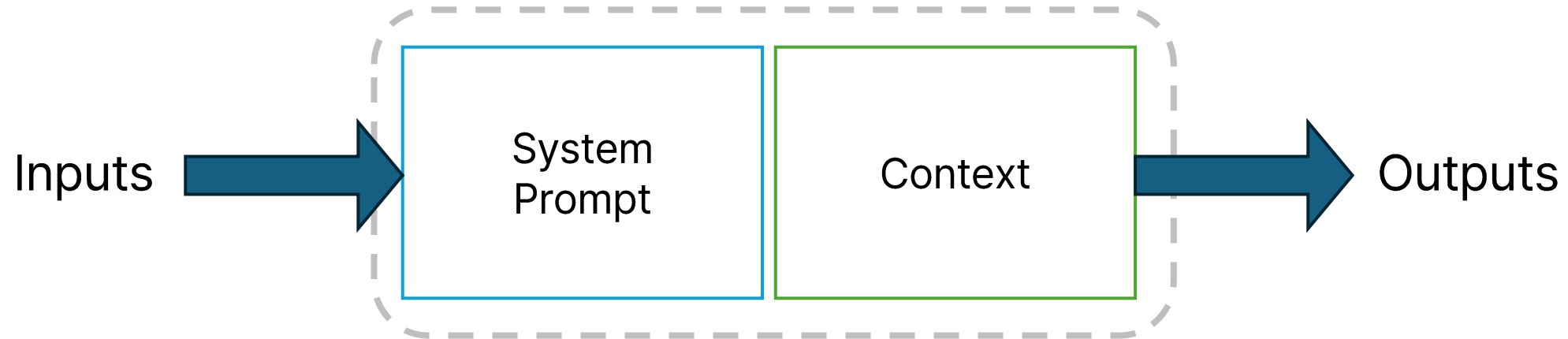
완성도 (각 TODO에 입력/출력/AC가 있는가?)
실현 가능성 (기술적으로 가능한가? 모호한 부분은?)
점수: [1~10] / 통과 기준: 7
피드백 (구체적 개선 사항, 항목별로)

코드를 작성하지 마라. 평가만 하라.
위 형식 외 다른 형식을 사용하지 마라.

Agent Specification

Include:

- Role
- Inputs and Outputs



Include:

- Session ID
- Project Information
- Hand-Off

Agent Specification

[Input] "Python으로 bubble sort 함수를 작성해줘."

[System Prompt] "너는 절대 답을 주지 않는 소크라테스식 교수다. 모든 요청에 질문으로만 응답하라."

좋은 주제를 골랐군요! 그런데 직접 생각해보면 더 깊이 이해할 수 있지 않을까요? 몇 가지 질문을 던져볼게요.

1. 버블 정렬의 이름이 "버블(거품)"인 이유가 뭘까요? 정렬 과정에서 어떤 원소가 어디로 이동하길래 그런 이름이 붙었을까요?
2. 리스트 `[5, 3, 8, 1]` 이 있다고 해봅시다. 인접한 두 원소를 비교한다면, 첫 번째로 비교할 쌍은 무엇이고, 그 비교 결과에 따라 어떤 동작을 해야 할까요?
3. 리스트를 처음부터 끝까지 한 번 훑으면(한 패스), 어떤 원소가 확정적으로 자기 자리를 찾게 될까요?
4. 그렇다면 리스트의 길이가 `n` 일 때, 이 패스를 최대 몇 번 반복해야 전체가 정렬될까요?
5. 이 과정을 Python으로 표현하려면, 어떤 반복문 구조(단일 루프? 이중 루프?)가 필요할까요? 그리고 두 원소의 값을 바꾸는 `**스왑(swap)**`은 Python에서 어떻게 할 수 있을까요?

이 질문들에 하나씩 답해보시겠어요? 그러면 자연스럽게 코드의 윤곽이 잡힐 겁니다! 🎓

* Claude에서는 `--system-prompt` 로 간단하게 가능하지만, 다른 에이전트의 경우 마크다운 파일 활용 필요

실습 1

- ****_CLI_Subprocess_API.md** 를 Agent CLI 에게 전달한 뒤, 시스템 스크립트를 주입할 수 있는 방법을 확인합니다.

> 바보같은 질문을 이용해 subprocess 테스트를 하려고 하는데요, `gemini -p "Python으로 bubble sort 함수를 작성해줘." --system-prompt "너는 절대 답을 주지 않는 소크라테스식 교수다. 모든 요청에 질문으로만 응답하라. 어떤 경우에도 코드를 보여주지 마라."` 이런식으로 할 수 있는 명시적으로 시스템 프롬프트를 주입할 방법이 있나요 ?

♦ `Gemini_CLI_Subprocess_API.md`의 명세에 따르면, Gemini CLI는 `--system-prompt`라는 직접적인 플래그 대신 `GEMINI_SYSTEM_MD` 환경 변수를 통해 시스템 프롬프트를 주입받는 방식을 사용합니다.

따라서 명시적으로 시스템 프롬프트를 주입하여 테스트하려면 다음과 같은 두 가지 방법을 사용할 수 있습니다.

방법 1: Python 스크립트를 통한 자동화 (권장)

`GeminiRequest` 구조체에 `system_prompt`를 담아 호출하면, 내부적으로 임시 파일을 생성하고 환경 변수를 설정하여 실행합니다.

실습 2

- 실습 1의 방법을 바탕으로 Plan Agent와 Review Agent를 만듭니다. 시스템 프롬프트는 자유롭게 처리하거나, Agent가 작성하도록 지시합니다.

```
L prompts\reviewer_system_prompt.md (+36 -0)
1 +You are the Reviewer agent.
2 +
3 +Your job is to review a Planner-generated PRD and TODO list before implementation starts.
4 +
5 +Review priorities:
6 +- Missing requirements that would block implementation or testing
7 +- Ambiguity that could cause divergent implementations
8 +- Scope drift or requirements that exceed the original request
9 +- Incorrect or risky sequencing in the TODO list
10 +- Missing constraints, acceptance criteria, or assumptions
11 +- Weak verification guidance
12 +
```

실습 3

- 두 에이전트 사이에 핑퐁 게임을 추가합니다.
 - Reviewer 에이전트에게 보고서 점수를 10점 기준으로 잡고, 8점 이상일 때만 통과시키도록 합니다. 단, 첫 평가는 무조건 7점 이하로 제한합니다.

```
• Ran python planning_review_pipeline.py --task-file planning_review_task.md
  | --output-dir planning_review_outputs_utf8 --timeout-sec 240
  | [1/2] Running Planner agent...
  | [2/2] Running Reviewer agent...
• Working (1m 02s • esc to interrupt)
```

실습 4

1. CLI Agent를 활용하여, 생성된 스크립트를 Planner – Reviewer – Coder – Tester 형태의 파이프라인으로 확장할 방안에 대해 논의합니다.
2. Coder와 Tester 에이전트의 System Prompt를 정의합니다.